420-301-VA: Web Services

Lab 7: Consuming Web Services with PHP and JavaScript Clients

Pravish Sainath
Computer Science & Technology
Vanier College

Introduction

In Labs 5 and 6, you built a production-ready RESTful backend API featuring routing, pagination, content negotiation, and structured error handling. However, an API does not exist in a vacuum; its purpose is to expose data and functionality to **Client Applications**.

In the real world, your web service will be consumed by two primary types of clients:

- **Server-to-Server Clients (Backend):** Another backend server (written in PHP, Python, or Java) making HTTP requests to your API to fetch data or trigger webhooks.
- **Browser-Based Clients (Frontend):** A Single-Page Application (SPA) or dynamic webpage using asynchronous JavaScript (`fetch`) to update the user interface without reloading the page.

In this lab, you will build both types of clients to consume your `library_db` API, implement client-side input validation, and gracefully display the structured `422` and `400` errors returned by your service.

Objectives

By the end of this lab, you should be able to:

- Configure your PHP API to handle **CORS Preflight (`OPTIONS`)** requests from web browsers.
- Implement a server-side HTTP client in PHP using the **cURL** extension (`curl_init`, `curl_setopt`, `curl_exec`).
- Build an interactive frontend client using HTML and modern JavaScript (`async`/`await` and the **Fetch API**).
- Dynamically render paginated collection data and pagination controls in the DOM.
- Implement client-side form validation to catch user errors before sending network requests.
- Parse and display structured backend validation errors (`422 Unprocessable Entity`) cleanly in the UI.

Instructions

- Ensure your Apache and MariaDB servers are running and that your `api.php` from Lab 6 is accessible at `http://localhost/api.php/books`.
- Create a new folder in your web root (e.g., `htdocs/client/`) to store the client files you will create in this lab (`php_client.php`, `index.html`, and `app.js`).
- In Part 4, you will transition to your Team Project repository to build the client consumption layer for your group's core resources.
- Document your code and capture screenshots of both your PHP client output and your interactive JavaScript web application (showing normal pagination and error states) for your lab submission.

1 Part 1: Server-Side HTTP Client in PHP (cURL)

1.1 Concept Review: Server-to-Server Communication & CORS

When one backend server needs to talk to another (for example, your PHP application calling the Stripe API for payments, or fetching data from a microservice), it acts as an HTTP client. In PHP, the industry-standard tool for making server-side HTTP requests is the **cURL library**.

Additionally, before we connect browser clients in Part 2, we must address a critical browser security feature: **CORS (Cross-Origin Resource Sharing)**. When a browser sends a `POST` request with a `Content-Type: application/json` header, it first sends an automatic `OPTIONS` request (called a "Preflight Request") to ask the server if this is allowed. If our middleware rejects `OPTIONS` requests or doesn't allow the `Content-Type` header, the browser will block the request!

Exercise 1: Preparing the API and Building a PHP Client

Step 1A: Upgrade `api.php` for Browser CORS Preflight

Open your `api.php` file from Lab 6. Right where you set your headers (before calling `runContentNegotiationMiddleware`), update the headers block to explicitly allow the `Content-Type` and `Accept` headers, and immediately return a `200 OK` if the browser sends an `OPTIONS` preflight request:

```
// --- Execute Pipeline in api.php ---
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header(
  "Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS"
);
header(
  "Access-Control-Allow-Headers: Content-Type, Accept, Authorization"
);

$method = $_SERVER['REQUEST_METHOD'];

// Handle CORS Preflight request from browsers immediately
if ($method === 'OPTIONS') {
    http_response_code(200);
    exit;
}
```

Step 1B: Creating a Reusable PHP cURL Client

Create a new file named `php_client.php` inside your `htdocs/client/` directory. We will write a helper function that encapsulates the 4-step cURL lifecycle: Initialize (`curl_init`), Configure (`curl_setopt`), Execute (`curl_exec`), and Close (`curl_close`).

Add the following code to `php_client.php`:

```
<?php
function callBookApi(
    string $method,
    string $url,
    ?array $payload = null
): array {
    // 1. Initialize cURL session
    $ch = curl_init($url);
```

```
    // 2. Set required headers for Lab 6 Content Negotiation
    $headers = [
        'Accept: application/json'
    ];

    // Configure basic options
    curl_setopt($ch, CURLOPT_CUSTOMREQUEST, $method);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

    // If sending data (POST/PUT/PATCH), attach JSON body
    if ($payload !== null) {
        $jsonData = json_encode($payload);
        $headers[] = 'Content-Type: application/json';
        curl_setopt($ch, CURLOPT_POSTFIELDS, $jsonData);
    }

    curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);

    // 3. Execute request and capture response & status code
    $rawResponse = curl_exec($ch);
    $statusCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    $curlError = curl_error($ch);

    // 4. Close cURL resource
    curl_close($ch);

    if ($curlError) {
        die("cURL Transport Error: " . $curlError);
    }

    return [
        'status' => $statusCode,
        'body' => json_decode($rawResponse, true)
    ];
}
```

Step 1C: Consuming the Service and Testing Error States in PHP

Still inside `php_client.php`, add the following test script below your function to consume your API and render the results in HTML. Notice how we test both a valid `GET` request and an intentionally invalid `POST` request to see how our PHP client handles the `422` response!

```php
$baseUrl = "http://localhost/api.php/books";

echo "<h1>PHP Server-Side API Client</h1>";

// --- TEST 1: Fetch Available Tolkien Books (GET) ---
echo "<h2>1. Fetching Available Tolkien Books</h2>";
$query = "?author=Tolkien&is_available=1";
$getResponse = callBookApi('GET', $baseUrl . $query);

if ($getResponse['status'] === 200) {
    $meta = $getResponse['body']['metadata'];
    $books = $getResponse['body']['data'];

    echo "<p>Found {$meta['total_items']} matching books ";
    echo "(Page {$meta['current_page']} of {$meta['total_pages']}):</p>";
    echo "<ul>";
    foreach ($books as $book) {
        $title = htmlspecialchars($book['title']);
        $author = htmlspecialchars($book['author']);
        echo "<li><strong>$title</strong> by $author</li>";
    }
    echo "</ul>";
} else {
    echo "<p style='color:red;'>Error fetching books!</p>";
}

// --- TEST 2: Triggering a 422 Validation Error (POST) ---
echo "<h2>2. Testing Server-Side Error Handling (Invalid POST)</h2>";
$badPayload = [
    "title" => "A", // Too short! Lab 6 requires >= 2 chars
    "author" => ""  // Empty!
];

$postResponse = callBookApi('POST', $baseUrl, $badPayload);
$status = $postResponse['status'];
echo "<p>HTTP Status Code Returned: <strong>$status</strong></p>";

if ($status >= 400) {
    $boxStyle = "background:#ffe6e6; border:1px solid red; padding:10px;";
    $errMsg = htmlspecialchars($postResponse['body']['error']);
    echo "<div style='$boxStyle'>";
    echo "<strong>API Error:</strong> $errMsg<br>";

    if (isset($postResponse['body']['details'])) {
        echo "<ul>";
        foreach ($postResponse['body']['details'] as $field => $issue) {
            $safeIssue = htmlspecialchars($issue);
            echo "<li><strong>$field:</strong> $safeIssue</li>";
        }
        echo "</ul>";
    }
    echo "</div>";
```

```
}
?>
```

Open your browser and navigate to ***`http://localhost/client/php_client.php`****. Verify that both the book list and the red validation error box render properly!*

2 Part 2: Browser Client in JavaScript (Fetch API)

2.1 Concept Review: Asynchronous JS and the Fetch Caveat

Modern web applications rarely reload the entire page when interacting with data. Instead, they use JavaScript's **`fetch()` API** alongside `async/await` to make background HTTP requests and dynamically update the HTML DOM.

Crucial Fetch Caveat: Unlike other HTTP libraries, **`fetch()` does NOT throw an exception on HTTP error status codes** (like `404`, `422`, or `500`)! It only throws an exception if the network itself fails (e.g., Wi-Fi disconnects or CORS blocks the request). Therefore, you must always check `response.ok` (which is `true` for status codes 200–299) or inspect `response.status` manually after parsing the JSON.

Exercise 2: Building an Interactive Frontend Application

Step 2A: Create the HTML User Interface

Create a file named `index.html` inside your `htdocs/client/` folder. Paste the following clean HTML structure. It includes filter controls, a table for our books, pagination buttons, an alert box for errors, and a form to create new books:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Library Manager Client</title>
  <style>
    body {
      font-family: sans-serif;
      max-width: 800px;
      margin: 20px auto;
      padding: 0 15px;
    }
    .box {
      border: 1px solid #ccc;
      padding: 15px;
      margin-bottom: 20px;
      border-radius: 5px;
    }
    .error-box {
      background: #ffe6e6;
      border-color: #cc0000;
      color: #990000;
      display: none;
    }
    .success-box {
      background: #e6ffe6;
      border-color: #009900;
      color: #006600;
      display: none;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin-top: 10px;
    }
```

```
    th, td {
      border: 1px solid #ddd;
      padding: 8px;
      text-align: left;
    }
    th { background-color: #f4f4f4; }
    .form-group { margin-bottom: 10px; }
    .form-group label { display: block; font-weight: bold; }
    .form-group input[type="text"] {
      width: 100%;
      padding: 6px;
      box-sizing: border-box;
    }
    .pagination {
      margin-top: 10px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
  </style>
</head>
<body>
  <h1>Library Web Service Client</h1>

  <!-- Feedback / Error Display Box -->
  <div id="feedbackBox" class="box error-box"></div>

  <!-- Filter & Collection Section -->
  <div class="box">
    <h2>Book Inventory</h2>
    <div>
      <input type="text" id="filterAuthor"
             placeholder="Filter by author (e.g. Tolkien)">
      <button onclick="applyFilter()">Search</button>
      <button onclick="resetFilter()">Reset</button>
    </div>

    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Title</th>
          <th>Author</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody id="bookTableBody"></tbody>
    </table>

    <div class="pagination">
      <button id="prevBtn" onclick="changePage(-1)">Previous</button>
      <span id="pageInfo">Page 1 of 1</span>
      <button id="nextBtn" onclick="changePage(1)">Next</button>
    </div>
  </div>

  <!-- Create Book Form -->
```

```
  <div class="box">
    <h2>Add a New Book</h2>
    <form id="addBookForm" novalidate>
      <div class="form-group">
        <label>Title:</label>
        <input type="text" id="bookTitle"
               placeholder="Enter book title">
      </div>
      <div class="form-group">
        <label>Author:</label>
        <input type="text" id="bookAuthor"
               placeholder="Enter author name">
      </div>
      <div class="form-group">
        <label>
          <input type="checkbox" id="bookAvailable" checked>
          Available for checkout
        </label>
      </div>
      <button type="submit">Create Book</button>
    </form>
  </div>

  <script src="app.js"></script>
</body>
</html>
```

Step 2B: Fetching Paginated Data in JavaScript

Create a file named `app.js` in the same `htdocs/client/` folder. Let's start by managing our pagination state and writing the `loadBooks()` function to call our API's `GET` endpoint and render the table rows:

```
const API_URL = 'http://localhost/api.php/books';
let currentPage = 1;
const limit = 4; // 4 items per page
let currentAuthorFilter = '';

// Helper to display error or success messages in the UI
function showFeedback(message, isError = true, detailsObj = null) {
  const box = document.getElementById('feedbackBox');
  box.style.display = 'block';
  box.className = isError ? 'box error-box' : 'box success-box';

  let html = `<strong>${message}</strong>`;
  if (detailsObj) {
    html += '<ul>';
    for (const [field, issue] of Object.entries(detailsObj)) {
      html += `<li><strong>${field}:</strong> ${issue}</li>`;
    }
    html += '</ul>';
  }
  box.innerHTML = html;
}

// 1. Fetch Paginated & Filtered Collection
async function loadBooks() {
  try {
    // Build URL with Query Parameters
    let url = `${API_URL}?page=${currentPage}&limit=${limit}`;
    if (currentAuthorFilter !== '') {
      const encoded = encodeURIComponent(currentAuthorFilter);
      url += `&author=${encoded}`;
    }

    const response = await fetch(url, {
      method: 'GET',
      headers: { 'Accept': 'application/json' }
    });

    const result = await response.json();

    if (!response.ok) {
      showFeedback(result.error || 'Failed to load books', true);
      return;
    }

    // Render Table Rows
    const tbody = document.getElementById('bookTableBody');
    tbody.innerHTML = ''; // Clear existing rows

    result.data.forEach(book => {
      const statusText = book.is_available ? 'Available' : 'Checked Out';
      const row = `<tr>
        <td>${book.id}</td>
```

```
        <td>${book.title}</td>
        <td>${book.author}</td>
        <td>${statusText}</td>
      </tr>`;
      tbody.innerHTML += row;
    });

    // Update Pagination Metadata Controls
    const meta = result.metadata;
    const totalPages = meta.total_pages || 1;
    document.getElementById('pageInfo').innerText =
      `Page ${meta.current_page} of ${totalPages} ` +
      `(Total: ${meta.total_items})`;

    const prevBtn = document.getElementById('prevBtn');
    const nextBtn = document.getElementById('nextBtn');
    prevBtn.disabled = (meta.current_page <= 1);
    nextBtn.disabled = (meta.current_page >= meta.total_pages);

  } catch (networkError) {
    showFeedback(
      'Network error: Could not reach the API server.',
      true
    );
  }
}

// Pagination and Filter UI Handlers
function changePage(direction) {
  currentPage += direction;
  loadBooks();
}

function applyFilter() {
  const input = document.getElementById('filterAuthor');
  currentAuthorFilter = input.value.trim();
  currentPage = 1; // Always reset to page 1 when filtering!
  loadBooks();
}

function resetFilter() {
  document.getElementById('filterAuthor').value = '';
  currentAuthorFilter = '';
  currentPage = 1;
  loadBooks();
}

// Load initial data when page opens
loadBooks();
```

Open `http://localhost/client/index.html` *in your browser. Test the "Next" and "Previous" buttons, and try searching for "Tolkien"!*

3 Part 3: Client-Side Validation and Error Handling

3.1 Concept Review: Defense in Depth

Why do we validate input on *both* the client and the server?

1. **Client-Side Validation (User Experience):** Checking inputs in JavaScript before sending the HTTP request gives the user instant feedback and saves unnecessary network traffic.
2. **Server-Side Validation (Security):** As we learned in Lab 6, a malicious user can bypass your HTML/JavaScript completely using Postman or `curl`. Therefore, the backend API must always enforce its own validation rules.

Exercise 3: Validating and Submitting the Form

Step 3A: Implementing Client Validation and POST Submission

Append the following form submission listener to the bottom of your `app.js` file. Notice how we perform client-side validation first, and then handle any `422` validation errors returned by the server:

```
const bookForm = document.getElementById('addBookForm');

bookForm.addEventListener('submit', async function(event) {
  event.preventDefault(); // Stop page reload

  const titleEl = document.getElementById('bookTitle');
  const authorEl = document.getElementById('bookAuthor');
  const availEl = document.getElementById('bookAvailable');

  const title = titleEl.value.trim();
  const author = authorEl.value.trim();
  const isAvailable = availEl.checked;

  // 1. Client-Side Input Validation
  const clientErrors = {};
  if (title.length === 0) {
    clientErrors.title = "Please enter a book title (Client Check).";
  }
  if (author.length === 0) {
    clientErrors.author = "Please enter an author name (Client Check).";
  }

  // If client validation fails, stop immediately and show errors!
  if (Object.keys(clientErrors).length > 0) {
    showFeedback("Client-Side Validation Failed", true, clientErrors);
    return;
  }

  // 2. Send POST Request to Web Service
  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
```

```
        title: title,
        author: author,
        is_available: isAvailable
      })
    });

    const result = await response.json();

    // 3. Handle Backend API Errors (e.g., 422 Unprocessable Entity)
    if (!response.ok) {
      const errMsg = `Server Error (${response.status}): ${result.error}`;
      showFeedback(errMsg, true, result.details || null);
      return;
    }

    // 4. Success! Clear form, show message, and refresh table
    showFeedback(`Success! Book created with ID: ${result.id}`, false);
    bookForm.reset();
    loadBooks(); // Refresh the table automatically

  } catch (error) {
    showFeedback("Network failure while submitting form.", true);
  }
});
```

Step 3B: Testing Both Validation Layers!

Let's see how our two validation layers work together:

1. **Trigger Client-Side Validation:** Leave both the Title and Author inputs completely blank and click "Create Book". Observe the red feedback box—it immediately catches the error in JavaScript (`Client Check`) without making a network request! (Open your browser's F12 Developer Tools → **Network** tab to verify no HTTP request was sent).

2. **Trigger Server-Side Validation (422):** Notice that our JavaScript check only verifies that the string is not empty (`length === 0`), whereas our Lab 6 backend requires at least **2 characters**! Type a single letter `"X"` into both Title and Author and click "Create Book". This passes the client check, sends the `POST` request over the network, and triggers the backend's `422 Unprocessable Entity` response! Verify that your UI cleanly parses and displays the backend's `details` object.

3. **Trigger Success (201):** Enter a valid book (e.g., "Neuromancer" by "William Gibson") and submit. Verify the green success banner appears and the table updates!

Step 3C: Your Turn - Expose `DELETE` Functionality to the Client

Currently, our table only displays books. Let's expose our API's `DELETE /api.php/books/{id}` endpoint to the user interface:

1. In `index.html`, add a fifth table header column: `<th>Action</th>`.

2. In `app.js` inside `loadBooks()`, add a "Delete" button inside a fifth `<td>` cell for each row:

```
<td><button onclick="deleteBook(${book.id})">Delete</button></td>
```

3. Write an `async function deleteBook(id)` in `app.js` that sends a `DELETE` request to `${API_URL}/${id}`.

4. Check `response.ok`. If successful, call `showFeedback("Book deleted!", false)` and call `loadBooks()` to refresh the table!

4 Part 4: Team Project Application

Exercise 4: Building Your Team Project's Client Layer

A core deliverable of your Team Project is proving that your web service can be consumed by external client applications. Meet with your team and integrate these client patterns into your project repository:

1. **Configure CORS on Your Project Backend:**
   - Update your project's Front Controller middleware to handle `OPTIONS` preflight requests and send the required `Access-Control-Allow-*` headers so browser clients can connect without being blocked.
2. **Build the Client Application (`/client` directory):**
   - Create a dedicated `client/` folder in your team repository.
   - Build a clean HTML/JavaScript dashboard that consumes your team's core endpoints.
   - For each team member's primary resource, the client application must demonstrate:
     (a) Fetching and rendering a **paginated and filtered collection** (`GET`) with working "Next/Previous" buttons and at least one filter input.
     (b) A form to create (`POST`) or update (`PUT`) a resource that implements **client-side validation** AND gracefully displays the backend's structured **422 validation errors** when triggered.
     (c) An interactive button in the UI to trigger a state change (`PATCH` or `DELETE`) on a specific resource ID.

References and Further Reading

- MDN Web Docs: Using the Fetch API: A complete guide to making asynchronous HTTP requests in JavaScript, setting headers, and checking `response.ok`.
- MDN Web Docs: Cross-Origin Resource Sharing (CORS): Understand why browsers send `OPTIONS` preflight requests when `application/json` payloads are transmitted.
- PHP Manual: Client URL Library (cURL): Official documentation on configuring server-to-server HTTP requests in PHP using `curl_setopt`.
- MDN Web Docs: Client-Side Form Validation: Best practices on combining frontend validation for UX with backend validation for security.